

# SIMATS ENGINEERING

## ASSESSMENT TOOL 1 – DESIGN TASK

**Course Name: Artificial Intelligence    Course Code: CSA17**

CO5: Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)

**Student Name:** Yuvasri.S

**Register Number:** 192511202

**Duration:** 1 Hour    **Total Marks:** 50

### CODE OF CONDUCT

I Yuvasri.S (192511202) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

**Signature of the student:** \_\_\_\_\_

## QUESTION 1 – HYBRID AI RECOMMENDATION ENGINE DESIGN

### 1. Problem Understanding & Requirement Analysis

The system recommends products, movies, courses or services by combining content-based and collaborative filtering. It must handle millions of user-item interactions, changing preferences, sparse ratings and low response time.

### 2. Objective

To build a hybrid recommendation engine that combines item similarity and behaviour learned from similar users, while intelligent agents continuously adapt recommendations using clicks, ratings, searches and feedback.

### 3. System Architecture Design

The architecture consists of data sources → distributed ingestion → PySpark RDD processing → content-based branch + collaborative-filtering branch → hybrid scoring/ranking → recommendation service → intelligent-agent feedback loop. User profiles and item metadata become feature RDDs; interaction data becomes collaborative-learning RDDs.

#### Data Flow

User/item data are loaded into RDDs. Invalid records are removed, features are generated, content and collaborative scores are calculated in parallel, scores are joined and combined, candidates are ranked, and recommendations are delivered. User feedback returns to the processing layer for the next update.

### 4. PySpark & RDD Operations

Use `textFile()/parallelize()` to create RDDs, `map()` for feature conversion, `filter()` for invalid data, `flatMap()` for item features, `reduceByKey()` for interaction aggregation, `join()` for profile/item information, `mapValues()` for scoring and `sortByKey()` for ranking. Cache frequently reused RDDs.

### 5. Intelligent Agent Integration

User Agent observes behaviour. Recommendation Agent generates candidates. Feedback Agent analyses clicks, skips and ratings. Learning Agent updates the content/collaborative weight. Coordinator Agent combines scores and selects the final recommendation list.

### 6. Scalability & Distributed Computing

RDD partitions allow user-item records to execute across many workers. `reduceByKey()` performs local aggregation before shuffle. Caching avoids repeated computation. Spark lineage supports recovery of lost partitions. The design can scale horizontally by adding executors.

### 7. Innovation & Creativity

Use  $\text{Score} = \alpha(\text{Content Score}) + (1-\alpha)(\text{Collaborative Score})$ , where  $\alpha$  changes according to recent feedback. New users receive stronger content-based recommendations, while users with sufficient history receive more collaborative recommendations.

### 8. Expected Result

The system provides personalized ranked recommendations, adapts to changing interests and can support very large user-item datasets with distributed processing.

## QUESTION 2 – SMART DISASTER DETECTION SYSTEM

### 1. Problem Understanding & Requirement Analysis

The system detects floods, earthquakes and related hazards from satellite information, IoT sensors, weather data and social-media reports. The main constraints are high volume, noisy observations, different data formats and the need for rapid warnings.

### 2. Objective

To build a distributed AI platform that fuses multi-source observations, detects abnormal patterns, estimates disaster severity and provides early warnings and response recommendations.

### 3. System Architecture Design

Architecture: satellite/sensor/social sources → distributed ingestion → PySpark preprocessing → feature extraction → data-fusion layer → risk/detection model → warning service. Intelligent agents monitor individual sources and coordinate final decisions.

## **Data Flow**

Source data are standardized, cleaned and transformed into regional features. Sensor statistics, image indicators and social signals are combined by geographic area. A risk score is calculated and high-confidence events are sent to the warning layer.

## **4. PySpark & RDD Operations**

Use `textFile()/parallelize()`, `filter()` for noise, `map()` for normalization and features, `flatMap()` for social-media terms, `union()` for compatible sources, `reduceByKey()` for regional statistics, `groupByKey()` for evidence collection and `join()` for combining risk features. `sortByKey()` prioritizes alerts.

## **5. Intelligent Agent Integration**

Sensor Agent monitors IoT data. Satellite Agent processes image-derived features. Social Agent identifies disaster-related messages. Fusion Agent compares evidence. Risk Agent estimates severity and confidence. Warning Agent issues alerts. Response Agent recommends evacuation or resource allocation.

## **6. Scalability & Distributed Computing**

Partition by geographic region or data source to process records in parallel. Broadcast variables can distribute small threshold/reference data. RDD lineage provides fault recovery. Regional partitioning reduces unnecessary network movement.

## **7. Innovation & Creativity**

Use an evidence-confidence score based on agreement among independent sources. A high-priority warning can require support from multiple reliable sources, reducing false alarms.

## **8. Expected Result**

The system provides early detection, confidence levels and prioritized alerts, helping emergency agencies identify affected areas and allocate resources quickly.

# **QUESTION 3 – AI-BASED FRAUD DETECTION FRAMEWORK**

## **1. Problem Understanding & Requirement Analysis**

The system must identify suspicious financial transactions from very large transaction streams. Fraud is usually rare and its patterns change over time, so the framework needs distributed anomaly detection, alert prioritization and continuous learning.

## **2. Objective**

To design a scalable fraud-detection system that calculates behavioural profiles, detects anomalies, correlates suspicious events and uses intelligent agents to continuously improve detection.

## **3. System Architecture Design**

Architecture: transaction streams → PySpark RDD preprocessing → feature engineering → customer/account profiles → anomaly scoring + rules → alert ranking → investigation. Agents exchange transaction, profile and investigation information.

## **Data Flow**

Transactions are cleaned and converted into behavioural features. Account-level statistics are aggregated, anomaly scores are generated, related events are correlated and suspicious cases are ranked. Confirmed fraud outcomes return to the learning agent.

## **4. PySpark & RDD Operations**

Create transaction RDDs with `map()`, filter invalid data, generate features with `map()`, `reduceByKey()/groupByKey()` for account statistics, `join()` customer/device information, `mapValues()` for risk scores and `sortByKey()` for alert priority. `persist()` can cache behavioural RDDs.

## **5. Intelligent Agent Integration**

Transaction Agent monitors events. Profile Agent maintains normal behaviour. Anomaly Agent calculates deviation. Investigation Agent correlates related transactions. Alert Agent assigns severity. Learning Agent updates thresholds/model parameters. Coordinator Agent combines evidence.

## **6. Scalability & Distributed Computing**

Partitioning by account or transaction region enables parallel computation. `reduceByKey()` minimizes shuffle for aggregations. Cached RDDs reduce repeated work. Lineage and checkpointing improve reliability for long-running pipelines. More worker nodes can increase throughput.

## **7. Innovation & Creativity**

Add a relationship-based risk score for transactions sharing devices, accounts, locations or beneficiaries. This helps detect coordinated fraud that may appear normal when transactions are examined individually.

## **8. Expected Result**

The framework produces ranked suspicious transactions and alerts, reduces manual investigation workload and adapts to emerging fraud patterns.

# **QUESTION 4 – AUTONOMOUS SUPPLY CHAIN OPTIMIZATION SYSTEM**

## **1. Problem Understanding & Requirement Analysis**

The system coordinates inventory, logistics and demand forecasting across suppliers, warehouses and stores. Challenges include huge historical datasets, changing demand, delivery delays, stock-outs and decentralized decisions.

## **2. Objective**

To build a distributed AI system that forecasts demand, detects inventory risk and selects cost-efficient supply and transportation decisions using cooperating agents.

## **3. System Architecture Design**

Architecture: sales/inventory/supplier/transport data → PySpark RDD preprocessing → demand forecasting + inventory analytics + logistics analytics → multi-agent coordination → optimized decision → execution and feedback.

## **Data Flow**

Sales and inventory records are cleaned and transformed. Demand is aggregated by product and period. Forecasts are combined with stock levels and supplier lead times. Agents compare alternatives and select feasible low-cost actions.

## **4. PySpark & RDD Operations**

Use `map()` for sales transformation, `filter()` for invalid transactions, `reduceByKey()` for demand totals, `groupByKey()` for historical sequences, `mapValues()` for statistics, `join()` to combine demand/inventory and `sortByKey()` to rank stock-out risk. Persist reused RDDs.

## **5. Intelligent Agent Integration**

Demand Agent forecasts demand. Inventory Agent monitors reorder risk. Supplier Agent evaluates capacity and lead time. Logistics Agent selects transport options. Warehouse Agent decides stock transfers. Coordinator Agent resolves conflicts and selects the lowest-cost feasible plan.

## **6. Scalability & Distributed Computing**

Partition by product, warehouse or region for parallel analysis. Avoid unnecessary `groupByKey()` where `reduceByKey()` is sufficient. Cache repeated forecasting data. Distributed executors process large sales histories and can scale horizontally.

## **7. Innovation & Creativity**

Use a multi-agent cost function considering purchase cost, transport cost, holding cost, stock-out penalty and delivery risk. Agents can negotiate resource allocation while respecting capacity constraints.

## **8. Expected Result**

The system improves demand visibility, reduces unnecessary inventory and supports faster, cost-efficient logistics decisions.

# **QUESTION 5 – PERSONALIZED LEARNING ANALYTICS PLATFORM**

## 1. Problem Understanding & Requirement Analysis

The platform analyzes learning behaviour and performance for thousands of learners using quiz scores, time spent, completed lessons, attempts and interaction frequency. It must personalize content while remaining scalable.

## 2. Objective

To design an AI learning platform that uses PySpark RDDs for large-scale learner analytics and intelligent agents for adaptive content selection, feedback and intervention.

## 3. System Architecture Design

Architecture: LMS/quiz/activity data → distributed ingestion → PySpark RDD preprocessing → learner-profile analytics → mastery/engagement prediction → content recommendation → intelligent-agent feedback loop.

### Data Flow

Learning events are cleaned and transformed into learner features. Scores and engagement measures are aggregated. Learner profiles are joined with course metadata. The recommendation agent selects the next resource, and feedback from the next activity updates the profile.

## 4. PySpark & RDD Operations

Use `textFile()`, `filter()`, `map()`, `reduceByKey()` for totals/averages, `groupByKey()` for learning histories, `join()` for learner/course data and `sortByKey()` for recommendation ranking. `cache()/persist()` can store frequently reused learner profiles.

## 5. Intelligent Agent Integration

Learner Agent monitors progress. Assessment Agent analyses quiz performance. Content Agent selects resources. Feedback Agent provides immediate guidance. Intervention Agent identifies learners needing support. Coordinator Agent combines decisions into an adaptive learning path.

## 6. Scalability & Distributed Computing

Partition by learner or course to process events in parallel. Distributed aggregations handle large activity logs. Caching reduces repeated profile calculations. RDD lineage supports fault recovery. Additional executors allow the platform to grow with learner volume.

## 7. Innovation & Creativity

Introduce a mastery-aware recommendation policy. The system estimates mastery and selects the next activity at an appropriate difficulty. After each assessment, the Feedback Agent updates the learning path.

## 8. Expected Result

The platform provides personalized learning paths, progress analytics and timely feedback while allowing educators to identify common difficulties and learners needing additional support.

## COMMON PYSPARK RDD OPERATIONS

**map()**: transforms every input record. **filter()**: removes records that do not satisfy a condition. **flatMap()**: creates zero or more output records from each input. **reduceByKey()**: combines values belonging to the same key and is efficient for distributed aggregation. **groupByKey()**: groups values by key. **join()**: combines two key-value RDDs. **sortByKey()**: orders key-value records. **union()**: combines compatible RDDs. Actions such as `count()`, `collect()` and `saveAsTextFile()` trigger execution. `cache()/persist()` improve repeated-access performance.

## OVERALL CO5 DESIGN PRINCIPLES

All five systems use distributed partitions, parallel execution, efficient aggregation, fault tolerance through RDD lineage, caching of reused data and horizontal scaling. Intelligent agents are assigned clear roles such as monitoring, prediction, coordination, feedback and decision-making. This separates large-scale data processing from adaptive AI decisions and demonstrates the integration required by CO5.

## CONCLUSION

The five designs demonstrate practical integration of PySpark, RDD operations, distributed computing and intelligent-agent architectures. The recommendation engine personalizes content, the disaster system fuses evidence for early warning, the fraud framework detects anomalies, the supply-chain system optimizes decentralized decisions, and the learning platform adapts educational content. Each design addresses scalability, data flow, fault tolerance, agent coordination and innovation, satisfying the major requirements of the assessment.

## SUBMISSION DETAILS

**Name:** Yuvasri.S

**Register Number:** 192511202

**Course:** Artificial Intelligence – CSA17

**Assessment Tool:** 1 – Design Task